

ARCHITECTURE DÉFINITIVE – DAR_Cyber

1. Statut du document

Ce document définit l'architecture de référence du projet DAR_Cyber. Il traduit les exigences du cahier des charges en choix techniques, composants, flux et contraintes de démonstration.

Il constitue une base stable pour le développement et la maintenance, tout en distinguant l'architecture cible et l'architecture de démonstration (Docker).

2. Objectif

L'objectif est de concevoir une application de supervision réseau capable de :

- détecter les équipements ;
- analyser le trafic ;
- identifier des anomalies ;
- générer des alertes ;
- produire des rapports.

L'architecture doit être :

- claire ;
- démontrable ;
- évolutive ;
- cohérente avec un MVP.

3. Stack technique

Backend :

- Python
- FastAPI

Base de données :

- PostgreSQL

Traitements :

- Redis
- Celery

Frontend :

- HTML / CSS / JS

Déploiement :

- Docker Compose

4. Principes d'architecture

L'architecture repose sur :

- séparation des responsabilités ;
- modularité ;
- centralisation de la logique métier ;
- traitement asynchrone ;
- traçabilité des actions ;
- simplicité pour le MVP.

Elle permet de concilier lisibilité, performance et évolutivité.

5. Architecture globale

Flux principal :

Endpoint → API → Base de données → Détection → Alertes → Interface

L'utilisateur accède uniquement via l'interface web.

Les données sont centralisées dans le serveur SOC.

6. Architecture Docker

Le système repose sur trois conteneurs :

- serveur-soc (API + DB + frontend)
- serveur-endpoint (collecte)
- serveur-attacker (simulation)

Cette architecture permet une démonstration reproductible.

7. Composants

API FastAPI :

- gestion des requêtes
- logique métier

PostgreSQL :

- stockage des données

Redis + Celery :

- traitement asynchrone

Frontend :

- visualisation

Agent endpoint :

- collecte données

8. Flux principal

1. Endpoint envoi données
2. API les reçoit
3. Stockage en base
4. Traitement
5. Génération alertes
6. Affichage interface

9. Modules backend

- auth
- telemetry
- scan
- analyse
- alerts
- reports
- audit

10. Modèle de données

- users
- assets
- events
- alerts
- logs
- exports

11. Exigences non fonctionnelles

- sécurité
- performance
- maintenabilité

- observabilité
- déployabilité

12. Conclusion

Cette architecture permet une solution cohérente, démontrable et évolutive, adaptée au projet DAR_Cyber et aux attentes d'un projet de niveau Master.